PROVISIONAL PATENT APPLICATION

TITLE OF THE INVENTION

Axiom Conflict Handling Protocol with Action Dispatch Interface (ACHP-ADI): A Control Architecture for Deterministic Management, Negative Cryptographic Attestation, and Execution Gating of Unresolved Probabilistic Decision States in Computing Systems

Inventor: Richard Davis

Filing Date: March 24, 2026

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a block diagram illustrating the four-layer system architecture pipeline of the ACHP-ADI control architecture, showing the Inference Engine, the Authority-State Derivation Layer, the Action Dispatch Interface (ADI), and the Action Executors, with non-bypassable constraints indicated.

FIG. 2 is a two-dimensional graph illustrating the decision region partitioning of the probability simplex into three mutually exclusive, exhaustive regions: Strong Decision, Weak/Flagged Decision, and Conflict (UNRESOLVED_ATTESTED), with threshold boundaries T_decide and T_sep indicated.

FIG. 3 is a flowchart illustrating the conflict handling flow, showing the diverging paths for action authorization and conflict condition handling, including negative attestation artifact generation, action blocking, and fallback invocation.

FIG. 4 is a diagram illustrating the non-instantiation enforcement boundary between the Inference Domain and the Execution Domain, showing hardware-enforced memory isolation, the authority-state derivation layer, zeroization mechanisms, and the structural absence of computational pathways from inference-domain data to action emission.

Title of the Invention

Axiom Conflict Handling Protocol with Action Dispatch Interface (ACHP-ADI): A Control Architecture for Deterministic Management, Negative Cryptographic Attestation, and Execution Gating of Unresolved Probabilistic Decision States in Computing Systems

Cross-Reference to Related Applications

This application claims priority to [U.S. Provisional Application No. _____, filed _____], the entire disclosure of which is incorporated herein by reference.

SECTION I — FIELD OF THE INVENTION

The present invention relates to control architectures for computing systems that process probabilistic inference outputs, and more particularly to a protocol that structurally prevents action execution when decision ambiguity exceeds defined thresholds, generates a cryptographic attestation of the negative condition that a decision could not be made, and gates all downstream action dispatch on the prior issuance of a valid decision-quality attestation by an independent authority layer.

The invention applies across domains including autonomous systems, robotic actuation, sensor fusion, distributed service architectures, API enforcement, and safety-critical decision pipelines.

SECTION II — BACKGROUND OF THE INVENTION

Computing systems increasingly rely on probabilistic inference models to drive downstream actions. These models produce probability distributions over candidate outputs, and the system selects an action based on the highest-ranked candidate. This pattern is pervasive: a classification model outputs class probabilities and the system acts on the argmax; a sensor fusion pipeline produces confidence-weighted assessments and the controller executes the highest-confidence response; an API gateway evaluates request risk scores and applies the top-ranked policy.

In all such systems, the inference-to-action pipeline has a structural deficiency: when the model's output distribution is ambiguous — when the top candidates are closely spaced and no single output commands sufficient confidence — the system nevertheless selects one output and dispatches the corresponding action. The system produces a confident-looking action even when the underlying decision is uncertain. This creates three categories of failure:

First, silent overconfidence. The downstream consumer of the decision (a robotic actuator, an API consumer, a human operator reviewing logs) has no indication that the decision was marginal. The action executes as if the model were certain.

Second, non-auditable ambiguity. After the fact, there is no record that the decision was ambiguous. The system log shows an input and an output, but not the fact that two or more candidates were nearly tied. Post-hoc forensic analysis cannot reconstruct the decision's uncertainty profile.

Third, non-bypassable forced execution. Even when the system's own metrics indicate insufficient separation between candidates, no structural mechanism exists to prevent action dispatch. The inference-to-action path is a pipeline, not a gated architecture. There is no

authority layer between the decision engine and the action executor that can independently halt execution based on decision quality.

Existing approaches address fragments of this problem but do not solve it as a structural whole.

Threshold-based classification rejection, established by Chow in 1970 and extended in subsequent work on selective classification, allows a classifier to abstain when the maximum posterior probability falls below a threshold. However, these methods produce a "reject" output within the classifier itself. They do not generate a structured attestation record of the rejection. They do not cryptographically sign or chain that record. They do not gate downstream action execution on the existence of such a record. They do not enforce a protocol-level invariant preventing override of the rejection. The rejection is an internal classifier behavior, not a system-level control mechanism.

Cryptographic audit trails for AI systems, including blockchain-based and hash-chained logging of inference decisions, have been proposed for regulatory compliance and forensic analysis. These systems log decisions that were made — they record the input, the model, and the selected output. They do not refuse to log a decision when ambiguity is present. They do not gate action execution on the content of the log. The logging is observational and post-hoc, not prescriptive and pre-execution.

Certified control architectures, such as the architecture proposed by Jackson et al. at MIT CSAIL for autonomous vehicles (2019–2020), employ an independent certifier that checks a controller-generated certificate of safety before permitting action. When the certificate check fails, a safe fallback controller takes over. This architecture teaches the structural separation of a decision engine from an independent authority that gates execution. However, the certifier evaluates physical safety conditions (obstacle geometry, lane compliance) based on sensor evidence. It does not evaluate the internal structure of a probability distribution for decision-

margin sufficiency. It does not generate a structured attestation record documenting that a decision could not be made. When the certifier rejects a certificate, no persistent record of the rejection is produced, no hash-chained audit trail of rejections is maintained, and no mechanism exists for an auditor to deterministically replay the rejection conditions and independently verify correctness.

Safety interlock systems in industrial and robotic contexts enforce pre-conditions for action execution. However, these interlocks are typically based on physical sensor states (e.g., door open/closed, pressure above/below threshold) or binary safety flags, not on analysis of a probability distribution's internal structure. They do not compute decision-margin metrics from inference outputs or produce structured attestation records of ambiguity.

Attested execution systems, including TEE-based remote attestation (Intel SGX, AMD SEV-SNP, Azure Attestation, IETF RATS), require cryptographic attestation as a precondition for code execution or secret release. The attestation token is a signed, structured record that must be verified before the relying party permits access. However, these systems attest a positive condition: that the execution environment is valid, that the code measurements match expected values, that the platform is trustworthy. They do not attest a negative condition: that a decision could not be made due to insufficient margin in a probability distribution. The attestation domain is environment integrity, not decision quality.

Zero-trust and policy-enforcement architectures in distributed systems enforce authorization requirements before action execution. However, these systems evaluate identity, credentials, and policy rules — not the probabilistic quality of a decision engine's output. They do not analyze probability distributions for insufficient separation or generate attestation records specific to decision ambiguity.

No existing system combines all of the following in an integrated protocol: (a) deterministic analysis of a probability distribution to detect insufficient decision margin using

fixed, interpretable thresholds; (b) structural refusal to emit any action-triggering output when ambiguity is detected; (c) generation of a structured attestation record of the negative condition that a decision could not be made, containing the full distribution, computed metrics, and applied thresholds in sufficient detail to enable deterministic replay of the decision boundary; (d) cryptographic signing of that record; (e) append-only, hash-chained, tamper-evident storage of signed records; (f) gating of all downstream action dispatch on the prior issuance of a valid decision-quality attestation by the authority layer, such that attestation is a mandatory precondition for — not a post-hoc log of — action execution; and (g) protocol-level enforcement that no downstream component may override a conflict determination to force an action.

SECTION III — SUMMARY OF THE INVENTION

The present invention provides a control architecture (ACHP-ADI) comprising two integrated subsystems:

The Axiom Conflict Handling Protocol (ACHP), which receives a probability distribution from a probabilistic inference engine, applies deterministic threshold comparisons to partition the distribution into decision regions, and — when the decision margin between the top-ranked candidates is insufficient — refuses to emit any action-triggering output and instead generates a cryptographically signed attestation record (Conflict Receipt) documenting the negative condition that a decision could not be made (UNRESOLVED_ATTESTED). The Conflict Receipt is a negative attestation: it attests the absence of a decision, not the presence of a valid one. It is structurally distinct from any classifier output, error report, or null response.

The Action Dispatch Interface (ADI), which structurally interposes between the ACHP evaluation layer and all downstream action executors. The ADI permits action dispatch only upon prior receipt of a valid decision-quality attestation from the Authority Layer. Attestation is a mandatory precondition for action execution, not an optional log or post-hoc record. No action path exists that bypasses the ADI. When the Authority Layer enters the conflict state, the ADI blocks all primary action paths and invokes a predefined fallback behavior. The ADI enforces the non-override invariant: no component may subsequently force an action for an input whose evaluation entered the conflict region.

The architecture converts probabilistic uncertainty into a verifiable control state through negative attestation. The Conflict Receipt is not a log entry — it is a cryptographically signed, hash-chained artifact that enables any auditor to perform deterministic replay of the decision boundary: to independently recompute the decision metrics from the recorded probability distribution and thresholds and verify that the conflict determination was mathematically correct and that no action was dispatched for the corresponding input.

The described architecture constitutes a practical application that improves the functioning of a computing system by altering how the system processes and represents data at a fundamental level. By eliminating the ability of the system to instantiate executable decision representations under defined conditions, the system reduces ambiguity-driven execution risk and constrains the operational state space of the machine. This is achieved through structural and hardware-enforced limitations on data representation and transformation, rather than through software policies applied after data creation.

SECTION IV — DEFINITIONS

UNRESOLVED_ATTESTED. A defined system control state indicating that the Authority Layer deterministically refused to produce an action-triggering output due to insufficient decision margin in the input probability distribution. This state is attested by a cryptographically signed Conflict Receipt. UNRESOLVED_ATTESTED is not a classification. It is not an error. It is not a null output. It is not a "reject" class label produced by the inference engine. It is a structurally defined control state that is external to the inference engine's output space, that activates the conflict handling path, and that blocks the primary action path. The Conflict State cannot be represented as, or reduced to, any output class of the inference engine.

Negative Attestation. A cryptographic attestation of a negative condition — specifically, that the system could not produce a decision of sufficient quality to authorize action execution. This is structurally distinct from positive attestation systems (such as TEE remote attestation, certificate-based safety validation, or code-integrity verification) which attest that a condition IS met. In ACHP-ADI, the negative attestation documents that a decision condition was NOT met, and that this non-meeting was the basis for blocking action execution. The Conflict Receipt is the embodiment of the negative attestation.

Authority Layer. The ACHP evaluation component that receives the probability distribution from the inference engine and independently determines whether the distribution exhibits sufficient decision quality to authorize action dispatch. The Authority Layer is structurally separate from the inference engine and from the action executors. It cannot be bypassed by either. The Authority Layer issues one of two outputs for each evaluation: an action authorization signal (positive attestation of sufficient decision quality) or a Conflict Receipt (negative attestation of insufficient decision quality). No evaluation may complete without one of these two outputs being issued to the ADI.

Action Dispatch Interface (ADI). The sole structural gate between the Authority Layer and all downstream action executors. The ADI accepts or rejects action dispatch requests based exclusively on the output of the Authority Layer. The ADI dispatches an action only upon prior receipt of an action authorization signal. No action may be dispatched without this prior authorization. When the Authority Layer enters the UNRESOLVED_ATTESTED state, the ADI blocks all primary action paths and invokes the fallback behavior. The ADI enforces the non-override invariant. There is no alternate execution path that circumvents the ADI.

Conflict Receipt (Attestation Record). A structured data record generated when the UNRESOLVED_ATTESTED state is entered. The Conflict Receipt is a negative attestation: it documents that a decision could not be made, not that one was made. It contains the full probability distribution, all computed metrics, applied thresholds, input identification, timestamps, device and lane identifiers, chain linkage, cryptographic signature, and the status indicator UNRESOLVED_ATTESTED. The receipt is the sole output of the conflict path. The receipt contains sufficient information to enable deterministic replay of the decision boundary — that is, an independent party can recompute the decision metrics from the recorded data and verify that the conflict region was correctly entered.

Conflict State. The system condition in which the Authority Layer has determined that the input probability distribution does not meet the criteria for any action-authorizing decision region. The Conflict State is structurally external to the inference engine's output space. It is not a label, class, or category produced by the inference engine. It is a control state of the authority-and-gating subsystem. The Conflict State triggers: (a) suppression of all action-triggering outputs; (b) generation and signing of a Conflict Receipt (negative attestation); (c) append of the signed receipt to the tamper-evident chain; (d) invocation of fallback behavior via the ADI; (e) blocking of all primary action paths for the corresponding input.

Decision Margin. A metric computed from the input probability distribution that quantifies the separation between the top-ranked candidate and its nearest competitor. The primary decision margin metric is the difference between the largest probability value and the second-largest probability value in the distribution. Alternate metrics quantifying decision ambiguity from the distribution (such as entropy-derived measures or other distributional separation measures) may serve as supplementary or alternative triggers under lane configuration.

Deterministic Replay of Decision Boundary. The process by which an independent auditor, using only the data contained in a Conflict Receipt, recomputes the decision metrics (including maximum probability, second-largest probability, and decision margin) from the recorded probability distribution, applies the recorded thresholds, and independently determines whether the conflict region was correctly entered. This replay is deterministic: given the same recorded distribution and thresholds, the result is always the same. This is distinct from event replay (which replays a sequence of actions) and from chain integrity verification (which verifies hash linkage and signatures). Decision boundary replay verifies the mathematical correctness of the conflict determination itself.

Replay Verification. The full audit process comprising: (a) chain integrity verification (signature and hash linkage checks); (b) deterministic replay of the decision boundary for each Conflict Receipt; and (c) confirmation that no action dispatch occurred for inputs whose evaluations entered the conflict region.

Lane. A named configuration scope that binds a specific set of thresholds, a fallback behavior policy, and a receipt retention policy to a particular operational domain or decision context.

Fallback Behavior. A lane-defined safe-mode action invoked by the ADI when the Conflict State is entered. The specific fallback behavior is not part of the ACHP-ADI core

protocol; it is a lane configuration parameter. The core protocol requires that a fallback is invoked, that the primary action path is blocked, and that no alternate execution path circumvents the fallback.

SECTION V — DETAILED DESCRIPTION OF THE INVENTION

5.1 System Architecture

The ACHP-ADI architecture comprises four structural layers arranged in a linear, non-bypassable pipeline. No alternate execution path exists that permits an inference engine output to reach an action executor without traversal through the Authority Layer and the ADI.

Layer 1 — Inference Engine. A probabilistic model (neural network, ensemble, Bayesian estimator, sensor fusion pipeline, or any system producing a probability distribution over candidate outputs) that receives raw input data and produces a probability distribution P over n candidate classes or actions, where n is at least 2. Each probability value is in [0, 1] and the values sum to 1 within a normalization tolerance. The Inference Engine has no direct connection to any action executor. The Inference Engine does not determine whether its own output is of sufficient quality for action authorization; that determination is the exclusive responsibility of the Authority Layer.

Layer 2 — Authority Layer (ACHP Evaluator). Receives the probability distribution P from the Inference Engine. Computes decision metrics from P. Applies fixed, lane-configurable thresholds to partition P into one of three mutually exclusive, exhaustive decision regions. For every evaluation, the Authority Layer produces exactly one of two output types and transmits it to the ADI: (a) an action authorization signal containing the selected candidate and a confidence indicator (positive attestation of sufficient decision quality), or (b) a Conflict Receipt attesting the UNRESOLVED_ATTESTED state (negative attestation that a decision could not be made). The Authority Layer has no ability to execute actions directly. No evaluation may pass from the Authority Layer to the ADI without one of these two outputs.

Layer 3 — Action Dispatch Interface (ADI). The sole structural gateway between the Authority Layer and all action executors. The ADI receives the output of the Authority Layer.

If the output is an action authorization signal, the ADI dispatches the corresponding action to the appropriate executor. If the output is a Conflict Receipt, the ADI blocks all primary action paths, appends the signed receipt to the tamper-evident chain, and invokes the lane-defined fallback behavior. The ADI enforces the non-override invariant: once a Conflict Receipt is received for a given input, no subsequent component or process may generate an action authorization for that input. There is no execution path that bypasses the ADI. Action Executors are structurally unreachable except through the ADI.

Layer 4 — Action Executors. Hardware actuators, API endpoints, notification systems, or any downstream consumers of action commands. Action Executors receive commands exclusively from the ADI. They have no direct access to the Inference Engine or the Authority Layer. No signal path permits an action executor to receive a command without the ADI having first received and validated an action authorization from the Authority Layer.

FIG. 1 (described): A block diagram showing four horizontally arranged layers. Left to right: Inference Engine outputs probability distribution P to Authority Layer. Authority Layer outputs either Action Authorization (positive attestation) or Conflict Receipt (negative attestation) to ADI. ADI outputs either Action Command to Action Executors or Fallback Signal to Fallback Handler. A vertical bar labeled "Tamper-Evident Chain" connects to the ADI, receiving Conflict Receipts. Dashed lines with "X" marks indicate: no direct path from Inference Engine to Action Executors; no direct path from Inference Engine to ADI; no path from any component to Action Executors that bypasses ADI.

5.2 Authority Layer — Decision Region Partitioning

The Authority Layer computes the following metrics from the input distribution P:

The maximum probability value (p_max): the largest value in P.

The second-largest probability value (p_second): the largest value in P excluding p_max. When two or more values are tied for the maximum, p_second equals p_max.

The decision margin: the difference between p_max and p_second. This metric quantifies the separation between the candidate that would be selected (the top-ranked output) and its nearest competitor. A small decision margin indicates that the top two candidates are nearly tied, and the selection of one over the other has low discriminative confidence.

Optionally, the global spread: the difference between p_max and the minimum value in P. This metric is recorded in attestation records for forensic analysis but is not used in region determination.

Optionally, any additional distributional ambiguity metric (such as Shannon entropy, Gini impurity, or other measure derived from the distribution) that quantifies the overall spread or concentration of probability mass. When enabled, these supplementary metrics may independently trigger the Conflict State.

The Authority Layer applies two fixed, lane-configurable thresholds:

A confidence threshold (T_decide), constrained to the open interval (0.5, 1). This threshold defines the minimum p_max required for a high-confidence decision.

A separation threshold (T_sep), constrained to the open interval (0, 1 - T_decide). This threshold defines the minimum decision margin required for a reduced-confidence decision when p_max falls below the confidence threshold.

These thresholds partition the probability simplex into three mutually exclusive, exhaustive regions:

Region 1 — Strong Decision. Condition: p_max is greater than or equal to T_decide. The top-ranked candidate commands sufficient confidence independently. The Authority

Layer emits an action authorization signal for the top-ranked candidate with a full-confidence indicator.

Region 2 — Weak Decision (Flagged). Condition: p_max is less than T_decide, AND the decision margin is greater than or equal to T_sep. The top-ranked candidate does not command full confidence, but the margin between it and the next candidate is sufficient to justify a provisional selection. The Authority Layer emits an action authorization signal for the top-ranked candidate with a mandatory low-confidence flag.

Region 3 — Conflict (UNRESOLVED_ATTESTED). Condition: p_max is less than T_decide, AND the decision margin is less than T_sep. Neither condition for action authorization is met. The Authority Layer enters the Conflict State, suppresses all action-triggering outputs, and generates a Conflict Receipt (negative attestation).

FIG. 2 (described): A two-dimensional plot. The horizontal axis represents the decision margin (0 to 1). The vertical axis represents p_max (0 to 1). A horizontal line at p_max = T_decide divides the plot into upper and lower regions. In the lower region, a vertical line at decision margin = T_sep divides it into left (Conflict) and right (Weak Decision). The upper region is labeled Strong Decision. The lower-left region is labeled Conflict (UNRESOLVED_ATTESTED — Negative Attestation). The lower-right region is labeled Weak Decision (Flagged).

Proof of exhaustiveness: For any probability distribution P with n at least 2, p_max has a definite value. Either p_max is at least T_decide (Strong Decision) or p_max is less than T_decide. In the latter case, the decision margin is a non-negative real number, so either it is at least T_sep (Weak Decision) or it is less than T_sep (Conflict). No distribution falls outside these three cases. The regions are pairwise disjoint by construction.

Boundary assignments: When p_max equals T_decide exactly, the distribution is assigned to Strong Decision. When the decision margin equals T_sep exactly, the distribution is assigned to Weak Decision.

5.3 Conflict Handling Procedure

When the Authority Layer determines that a distribution P enters the Conflict region for an evaluation associated with input data having cryptographic hash H:

Step 1. The Authority Layer suppresses all action-triggering outputs. No candidate label, class identifier, action code, or any other output that could be interpreted as a decision is emitted. The Conflict State is structurally external to the inference engine's output space: the inference engine does not produce a "conflict" or "reject" label. The absence of a decision is determined by the Authority Layer, not by the inference engine.

Step 2. The Authority Layer constructs a Conflict Receipt (negative attestation). The receipt is a structured data record documenting that a decision could not be made. It contains, at minimum: the full probability distribution P; the computed p_max, p_second, and decision margin values; the applied T_decide and T_sep thresholds; a cryptographic hash of the raw input data; a monotonic local timestamp and a UTC timestamp (or an unavailability indicator); a unique device identifier and lane identifier; the cryptographic hash of the immediately preceding receipt in the chain (or a genesis indicator for the first receipt); and the status indicator UNRESOLVED_ATTESTED. The receipt contains sufficient information to enable deterministic replay of the decision boundary: any party possessing the receipt can recompute p_max, p_second, and the decision margin from the recorded probability distribution, apply the recorded thresholds, and independently verify that the conflict region was correctly entered.

Step 3. The Authority Layer cryptographically signs the receipt using the device's private key via an asymmetric signature algorithm. The signature covers a cryptographic hash of the receipt's content serialized in a canonical, deterministic order. If the signing module is

unavailable, the receipt is stored with a pending-signature indicator and signing is retried upon module availability. Unsigned receipts are flagged as unverified in any audit traversal.

Step 4. The signed Conflict Receipt is transmitted to the ADI. This is the sole output of the conflict path. No action authorization signal accompanies or follows this transmission for the same input.

Step 5. The ADI, upon receiving a Conflict Receipt, blocks all primary action paths for the corresponding input. This blocking occurs prior to and instead of any action dispatch. The ADI appends the signed receipt to the local tamper-evident chain. The chain is stored in non-volatile memory. Each receipt references the preceding receipt's hash, forming an append-only, hash-linked sequence. Receipts may only be appended; no deletion,
modification, or reordering is permitted.

Step 6. The ADI invokes the lane-defined fallback behavior.

Step 7. The system continues in fallback mode until the next evaluation cycle.

FIG. 3 (described): A flowchart with two paths diverging from the Authority Layer decision point. Left path (Normal): Authority Layer emits Action Authorization (positive attestation) → ADI validates authorization → ADI dispatches Action Command → Action Executor performs action. Right path (Conflict): Authority Layer emits Conflict Receipt (negative attestation) → ADI blocks all action paths → ADI appends receipt to chain → ADI invokes fallback → system enters fallback mode. Both paths converge at "Await Next Evaluation." A label on the ADI node reads "No action without prior authorization from Authority Layer."

5.4 Action Dispatch Interface — Execution Gating

The ADI is the sole structural gateway between the Authority Layer and all action executors. No alternate execution path exists. Its operation is governed by four rules:

Rule 1 — No action without prior authorization. The ADI dispatches an action command to an action executor only upon prior receipt of a valid action authorization signal from the Authority Layer for the corresponding evaluation. The authorization must be received before the action is dispatched. No other component, process, signal path, or timing sequence may cause the ADI to dispatch an action. Attestation of decision quality by the Authority Layer is a mandatory precondition for action execution, not an optional verification or post-execution log.

Rule 2 — Conflict blocks action. Upon receipt of a Conflict Receipt (negative attestation) from the Authority Layer, the ADI blocks all primary action paths for the corresponding evaluation. The ADI does not attempt to interpret, override, or resolve the conflict. The receipt is appended to the chain and the fallback is invoked. The blocking is immediate and unconditional.

Rule 3 — Non-override enforcement. Once the ADI has received a Conflict Receipt for an input with hash H, no subsequent signal — from the Authority Layer, from any downstream component, from any external process — may cause the ADI to dispatch an action command for input hash H. Conformance with this rule is verifiable by deterministic replay: if the receipt chain contains a Conflict Receipt for input hash H, no action dispatch log entry for input hash H may exist.

Rule 4 — No alternate path. No execution path exists in a conforming implementation that permits an inference engine output to reach an action executor without passing through the ADI. Action Executors are structurally unreachable except through the ADI.

5.5 Attestation Record Structure

The Conflict Receipt contains the following required fields, specified in canonical order to enable deterministic hashing:

Schema version identifier. Cryptographic hash of the raw input data. Full probability distribution as an ordered array of floating-point values. Ordered labels corresponding to each probability value. Number of candidates. Maximum probability value and its index. Second-largest probability value and its index. Decision margin (p_max minus p_second). Global spread (p_max minus minimum probability), recorded for forensic diagnostics only. Applied confidence threshold (T_decide). Applied separation threshold (T_sep). Decision region indicator (CONFLICT). Status indicator (UNRESOLVED_ATTESTED). Lane identifier. Device identifier. Monotonic local timestamp. UTC timestamp (or unavailability indicator). Hash of the preceding receipt in the chain (or genesis indicator). Hash of the current receipt's content. Cryptographic signature over the receipt hash. Signature algorithm identifier. Signature status (signed or pending).

The receipt structure is designed to satisfy the requirement for deterministic replay of the decision boundary. Specifically, the recorded probability distribution and thresholds are sufficient for any party to independently recompute all decision metrics and verify that the conflict region was correctly entered without access to any information beyond the receipt itself.

Optional fields include: entropy threshold and computed entropy (when the entropy extension is active); supplementary distributional ambiguity metric values; trigger source indicator; threshold configuration source identifier; human attestation annotation (post-emission, linked to the original receipt hash, non-retroactive); compression indicator; and lane-specific metadata.

5.6 Audit and Replay Verification

An auditor, whether local or remote after chain synchronization, performs the following verification:

Step 1. Traverse the receipt chain from genesis to the target receipt.

Step 2. Verify each cryptographic signature using the corresponding device's public key.

Step 3. Verify hash chain linkage: each receipt's preceding-receipt-hash field must match the receipt hash of the immediately prior entry.

Step 4. For each Conflict Receipt, perform deterministic replay of the decision boundary: extract the recorded probability distribution and thresholds; recompute p_max, p_second, and the decision margin; confirm that the recomputed values satisfy the conflict region conditions (p_max less than T_decide AND decision margin less than T_sep). This replay is deterministic and reproducible: the same recorded data always produces the same verification result.

Step 5. Confirm that no action dispatch log entry exists for the same input hash.

This process verifies three independent properties: chain integrity (Steps 1–3), mathematical correctness of each conflict determination (Step 4), and absence of unauthorized action execution (Step 5). The combination of these three properties — not any one in isolation — constitutes full replay verification.

Replay verification confirms internal consistency of recorded data. It does not independently verify that the recorded thresholds were the thresholds actually applied at decision time. Binding decision-time thresholds to receipt content requires trusted execution or hardware attestation, which is outside the scope of the ACHP-ADI core protocol and is disclosed as a known limitation.

SECTION VI — INVARIANTS

The following invariants define the correctness conditions for any conforming implementation of ACHP-ADI.

INV-1 (No Forced Output). No action-triggering output is emitted from any execution path that has entered the Conflict region for that evaluation. The sole output of the conflict path is the Conflict Receipt (negative attestation). No candidate label, class identifier, action code, "reject" class, or any other classifier output is emitted.

INV-2 (One Receipt Per Conflict). Every entry into the Conflict region produces exactly one Conflict Receipt.

INV-3 (Signed Attestation). Every Conflict Receipt is cryptographically signed using the device's private key, or is flagged as signature-pending with a queued retry.

INV-4 (Append-Only Chain). Signed receipts are stored in an append-only, hash-linked sequence in non-volatile memory. No receipt may be removed, modified, or reordered.

INV-5 (Non-Override). No downstream component, post-processor, wrapper, or external process may override a Conflict determination to produce an action dispatch for the same input. Conformance is verifiable by deterministic replay of the decision boundary: if the chain contains a Conflict Receipt for input hash H, no action dispatch log for input hash H may exist.

INV-6 (No Action Without Prior Attestation). The ADI dispatches an action command only upon prior receipt of a valid action authorization signal from the Authority Layer for the corresponding evaluation. Attestation of decision quality is a mandatory precondition for action execution. No other path to action execution exists in a conforming implementation. No action may be dispatched before, without, or in the absence of the Authority Layer's determination.

INV-7 (Conflict Forces Fallback). When the Conflict State is entered, the ADI invokes the lane-defined fallback behavior. The fallback is not optional. The specific fallback behavior is a lane configuration parameter; its invocation is a protocol requirement.

INV-8 (No Alternate Execution Path). No execution path exists in a conforming implementation that permits an inference engine output to reach an action executor without traversal through the Authority Layer and the ADI. Action Executors are structurally unreachable except through the ADI.

INV-9 (Conflict State Externality). The Conflict State is structurally external to the inference engine's output space. The inference engine does not produce, emit, or participate in the determination of the Conflict State. No output label, class, or category of the inference engine may represent or substitute for the Conflict State. The Conflict State is determined exclusively by the Authority Layer based on analysis of the probability distribution's internal structure.

SECTION VII — EMBODIMENTS

7.1 Embodiment 1 — AI Classification System (Agricultural Monitoring)

An edge computing device (comprising a processor, non-volatile storage, a cryptographic module, and a local inference engine) monitors agricultural conditions. The inference engine is a convolutional neural network classifying sensor data (acoustic, thermal, weight, visual) into four candidate conditions: Healthy, Pest Risk, Environmental Stress, and Equipment Fault.

Lane configuration: confidence threshold = 0.90; separation threshold = 0.15.

Worked Example: The inference engine produces P = [0.45, 0.42, 0.08, 0.05] for the four candidates respectively.

Authority Layer computation: p_max = 0.45 (Healthy). p_second = 0.42 (Pest Risk). Decision margin = 0.45 - 0.42 = 0.03. Global spread = 0.45 - 0.05 = 0.40 (diagnostic only).

Region evaluation: p_max = 0.45 is less than T_decide = 0.90. Decision margin = 0.03 is less than T_sep = 0.15. The distribution enters the Conflict region.

Authority Layer action: Suppresses all candidate outputs. Generates a Conflict Receipt (negative attestation) containing the full distribution [0.45, 0.42, 0.08, 0.05], all computed metrics, applied thresholds, input hash, timestamps, device and lane identifiers, chain linkage, and status = UNRESOLVED_ATTESTED. Signs the receipt. Transmits to the ADI.

ADI action: Blocks all primary action paths (no automated treatment actuation, no status-change command) prior to and instead of any action dispatch. Appends the signed receipt to the local chain. Invokes fallback: activates a local alert indicator (LED strobe, dashboard flag) for operator inspection. Maintains passive telemetry logging.

Deterministic replay: An auditor receiving the receipt extracts P = [0.45, 0.42, 0.08, 0.05], T_decide = 0.90, T_sep = 0.15. Recomputes p_max = 0.45, p_second = 0.42, decision margin = 0.03. Confirms: 0.45 < 0.90 AND 0.03 < 0.15. Conflict region correctly entered. Verifies no action dispatch log for the corresponding input hash.

7.2 Embodiment 2 — Robotic Actuator Control (Path Planning Under Sensor Conflict)

A robotic system comprises multiple perception sensors (lidar, stereo camera, ultrasonic) whose outputs are fused by a probabilistic model to classify terrain or obstacle conditions. The classification drives actuator commands: proceed, detour left, detour right, halt.

Lane configuration: confidence threshold = 0.85; separation threshold = 0.10.

Worked Example: The fusion model produces P = [0.38, 0.35, 0.17, 0.10] for [Proceed, Detour Left, Detour Right, Halt].

Authority Layer computation: p_max = 0.38 (Proceed). p_second = 0.35 (Detour Left). Decision margin = 0.38 - 0.35 = 0.03.

Region evaluation: p_max = 0.38 is less than T_decide = 0.85. Decision margin = 0.03 is less than T_sep = 0.10. Conflict region entered.

Authority Layer action: Suppresses all actuator commands. Generates and signs a Conflict Receipt (negative attestation). Transmits to the ADI.

ADI action: Blocks all motor commands prior to any actuation. Appends receipt to chain. Invokes fallback: commands motor halt, assumes static safe pose, activates teleoperation beacon, awaits manual override.

Deterministic replay: Auditor extracts P = [0.38, 0.35, 0.17, 0.10], T_decide = 0.85, T_sep = 0.10. Recomputes decision margin = 0.03. Confirms 0.38 < 0.85 AND 0.03 < 0.10. Conflict correct. No motor command log for corresponding input hash.

7.3 Embodiment 3 — Distributed System / API Enforcement (Multi-Factor Risk Assessment)

An API gateway receives incoming requests and evaluates them against a probabilistic risk model that classifies each request into risk tiers: Approve, Approve with Monitoring, Escalate to Human Review, and Deny.

Lane configuration: confidence threshold = 0.95; separation threshold = 0.20.

Worked Example: The risk model produces P = [0.40, 0.38, 0.15, 0.07] for [Approve, Approve with Monitoring, Escalate, Deny].

Authority Layer computation: p_max = 0.40 (Approve). p_second = 0.38 (Approve with Monitoring). Decision margin = 0.40 - 0.38 = 0.02.

Region evaluation: p_max = 0.40 is less than T_decide = 0.95. Decision margin = 0.02 is less than T_sep = 0.20. Conflict region entered.

Authority Layer action: Suppresses all API response actions. Generates and signs a Conflict Receipt (negative attestation) containing the full risk distribution, applied policy thresholds, request metadata hash, and chain linkage. Transmits to the ADI.

ADI action: Blocks the API response pipeline prior to any response transmission. Appends receipt to chain. Invokes fallback: returns a standardized "decision pending" response to the requester with a receipt reference identifier, routes the request to a human review queue, and logs the conflict event for compliance audit.

Deterministic replay: Regulatory auditor extracts P and thresholds from receipt, recomputes decision margin, confirms conflict was correctly triggered, verifies no auto-approval log entry for the corresponding request hash.

SECTION VIII — CLAIMS

Independent Claim 1 (Method)

A computer-implemented method for controlling action execution in a system that processes probabilistic inference outputs, comprising:

(a) receiving, at an authority-state derivation layer that is structurally separate from aprobabilistic inference engine and from all action executors, a probability distribution produced by the inference engine over two or more candidate outputs;

(b) deriving, at the authority-state derivation layer, an execution-authority state fromthe probability distribution by computing at least a decision-quality metric that quantifies the separation between the highest-ranked candidate and the next-highest-ranked candidate, and

comparing the decision-quality metric and the maximum probability value in the distribution against fixed, lane-configurable thresholds;

(c) when the maximum probability value is below a first threshold and thedecision-quality metric is below a second threshold, determining that no execution-authority state authorizing action exists for the corresponding input, thereby entering a conflict condition in which no decision output exists and no executable decision object, intermediate action representation, candidate label, reject token, reject class output, inference-produced symbolic output, or any other data structure usable to initiate, derive, or approximate an action is instantiated for the corresponding input, wherein the conflict condition is not an output of the inference engine, is not a classification label, is not a reject class, is not any inference-produced symbolic or categorical output, and is not representable by any output of the inference engine regardless of how the inference engine's output space is defined, modified, or extended, and wherein no output of the inference engine constitutes or substitutes for the conflict condition, such that no executable or classifiable representation of a decision is formed for the corresponding input, and wherein no downstream component may derive, compute, or reconstruct an action from the probability distribution, any subset of the probability distribution, or any feature, metric, or metadata derived from the probability distribution, and wherein said prohibition applies across all stages of computation, including but not limited to transient, intermediate, cached, register-level, speculative, or ephemeral machine states, such that no executable, classifiable, or derivable decision representation is ever formed, materialized, stored, transmitted, or recoverable, even momentarily, within any component of the system;

(d) upon entering the conflict condition, generating a negative attestation artifact thatdocuments that no decision could be produced, the negative attestation artifact containing at least: the full probability distribution, the computed decision-quality metric, the maximum probability value, the applied thresholds, and an identification of the input, wherein the negative attestation artifact is sufficient, independently and without access to the inference engine, to

deterministically reconstruct the decision-state transition that caused the conflict condition to be entered, and wherein the negative attestation artifact constitutes the exclusive persistent representation of the system state for the corresponding input such that no parallel, duplicate, cached, or derived state representation for the same input may exist, persist, or be used to influence the non-bypassable execution interface, and any such parallel state is structurally inadmissible for action derivation;

(e) cryptographically signing the negative attestation artifact;

(f) appending the signed negative attestation artifact to a tamper-evident, append-only,ordered log stored in non-volatile memory, wherein each entry references the cryptographic hash of the preceding entry;

(g) transmitting the negative attestation artifact to a non-bypassable execution interfacethat is interposed between the authority-state derivation layer and all action executors as the sole path through which action commands may reach any action executor, the execution interface operating as a state-driven emitter that produces action commands exclusively by mapping attested execution-authority states to action commands through a fixed correspondence, the execution interface having no input path that receives, carries, or is derived from any decision object, candidate action, inference engine output, probability distribution, or data derived therefrom, such that no data path exists from inference outputs to action emission within or through the execution interface, the execution interface being structurally absent the computational primitives over inference-domain data necessary to derive, infer, reconstruct, approximate, or compute any action from non-attested data, from the probability distribution, from any subset or transformation of the probability distribution, from any inference engine output, or from any feature, signal, or metadata not contained within an attested execution-authority state, wherein the structural absence of said primitives is enforced by at least one of: (i) the execution interface lacking hardware arithmetic and statistical processing units capable

of operating on inference-domain data types, or (ii) the execution interface operating within an address space from which all inference-domain data is structurally excluded by hardware-enforced memory isolation such that no sequence of operations available to the execution interface can access, load, or operate upon inference-domain data; and

(h) at the non-bypassable execution interface, blocking all action execution paths forthe corresponding input and invoking a predefined fallback behavior, wherein actions are not derived from inference engine outputs but are derived only from attested execution-authority states issued by the authority-state derivation layer, wherein blocking and fallback invocation occur prior to and instead of any action dispatch for the corresponding input, and wherein no execution path, side-channel, duplicate pipeline, shadow process, or alternative signal route exists through which an inference engine output, a probability distribution, or any data derived therefrom may reach an action executor or cause action execution without traversal through the authority-state derivation layer and the non-bypassable execution interface.

Independent Claim 2 (System)

A computing system comprising:

a probabilistic inference engine that produces a probability distribution over two or more candidate outputs; an authority-state derivation layer that is structurally separate from the inference engine and from all action executors, the authority-state derivation layer configured to receive the probability distribution and to derive an execution-authority state therefrom by computing at least a decision-quality metric quantifying the separation between the highest-ranked candidate and the next-highest-ranked candidate and comparing the decision-quality metric and the maximum probability value against fixed thresholds, and further configured to, when no execution-authority state authorizing action exists, generate a negative attestation artifact documenting that no decision could be produced, the artifact containing the probability

distribution, the decision-quality metric, and the applied thresholds in sufficient detail to deterministically reconstruct the decision-state transition independently and without access to the inference engine; a cryptographic module configured to sign the negative attestation artifact; non-volatile memory storing a tamper-evident, append-only, ordered log of signed negative attestation artifacts; a hardware-enforced memory isolation mechanism that partitions inference-domain data into a memory region that is not addressable by the execution interface, such that attempted access by the execution interface to said region is prevented by a hardware-level fault; and

a non-bypassable execution interface interposed between the authority-state derivation layer and one or more action executors as the sole path through which action commands may reach any action executor, the execution interface operating as a state-driven emitter that produces action commands exclusively by mapping attested execution-authority states to action commands through a fixed correspondence, the execution interface having no input path that receives or carries any decision object, candidate action, inference engine output, or probability distribution, the execution interface being structurally absent the computational primitives over inference-domain data necessary to derive, infer, reconstruct, approximate, or compute any action from non-attested data or from any inference engine output, the structural absence of said primitives being enforced by at least one of: (i) the execution interface comprising a fixed-function logic block that lacks a general-purpose arithmetic logic unit and floating-point unit, or (ii) the execution interface operating within an address space from which all inference-domain data is excluded by the hardware-enforced memory isolation mechanism, the execution interface configured to dispatch an action only upon receipt of an attested execution-authority state authorizing that action, and to block all action paths and invoke a fallback behavior upon receipt of a negative attestation artifact; wherein no execution path, side-channel, duplicate pipeline, shadow process, or alternative signal route permits an inference engine output to reach an action executor or cause action execution without traversal through the authority-state derivation layer

and the non-bypassable execution interface, wherein no data path exists from inference outputs to action emission within or through the execution interface, wherein actions are derived only from attested execution-authority states and not from inference engine outputs directly, and wherein the negative attestation artifact constitutes the exclusive persistent representation of the system state for a conflicted input such that no parallel or derived state may influence the execution interface.

Independent Claim 3 (Non-Transitory Medium)

A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

Claim 4 — Conflict State Externality and Reject-Class Exclusion (depends on Claim 1)

The method of Claim 1, wherein the conflict condition is structurally external to the output space of the inference engine such that no modification, addition, extension, or reconfiguration of the inference engine's output classes, labels, categories, or symbolic outputs — including but not limited to the addition of a reject class, an abstention output, an uncertainty indicator, or any other inference-produced token — can represent, produce, or substitute for the conflict condition, and wherein no component receiving inference engine outputs may use those outputs to derive, simulate, or approximate the conflict condition or any execution-authority state.

Claim 5 — Distributional Ambiguity Trigger (depends on Claim 1)

The method of Claim 1, further comprising computing a distributional ambiguity metric from the probability distribution, and entering the conflict condition when the distributional ambiguity metric exceeds a third threshold regardless of the decision-quality metric.

Claim 6 — Ensemble Aggregation (depends on Claim 1)

The method of Claim 1, wherein the probability distribution is computed as an aggregate of outputs from a plurality of inference models before the authority-state derivation layer derives the execution-authority state.

Claim 7 — Deterministic Replay Procedure (depends on Claim 1)

The method of Claim 1, further comprising deterministic replay, wherein an independent party, using only the data contained in the negative attestation artifact and without access to the inference engine, the authority-state derivation layer, or any internal system state, recomputes the decision-quality metric and the maximum probability value from the recorded probability distribution, applies the recorded thresholds, independently determines whether the conflict condition was correctly entered, and confirms that no action dispatch occurred for the corresponding input.

Claim 8 — Non-Override Invariant (depends on Claim 1)

The method of Claim 1, wherein the non-bypassable execution interface enforces a non-override invariant such that once a negative attestation artifact has been received for a given input, no subsequent signal from any component, process, or external system causes the execution interface to dispatch an action for that input, and wherein any parallel, cached, or derived state representation for the same input that was not produced by the authority-state derivation layer is structurally inadmissible at the execution interface and cannot influence action dispatch.

Claim 9 — Extended Artifact Fields (depends on Claim 1)

The method of Claim 1, wherein the negative attestation artifact further comprises: a cryptographic hash of the raw input data; a monotonic local timestamp; a UTC timestamp or unavailability indicator; a device identifier; a lane identifier specifying the operational configuration; a hash of the immediately preceding entry in the ordered log; and a signature status indicator that is set to a pending state when the cryptographic module is unavailable, with signing retried upon module availability, and wherein the non-bypassable execution interface blocks action execution and invokes fallback regardless of whether signing has completed.

Claim 10 — Edge and Offline Operation (depends on Claim 2)

The system of Claim 2, wherein the computing system operates in an offline-first mode on an edge computing device, the tamper-evident ordered log is stored in device-local non-volatile memory, and the system periodically synchronizes the log to a remote audit repository upon network availability.

Claim 11 — Hardware Secure Element (depends on Claim 2)

The system of Claim 2, wherein the cryptographic module comprises a hardware secure element or trusted platform module storing a device-unique private key, and wherein the negative attestation artifact is signed using the device-unique private key.

Claim 12 — Platform Integrity Binding (depends on Claim 1)

The method of Claim 1, further comprising binding the negative attestation artifact to a platform integrity measurement provided by a hardware attestation mechanism, thereby linking the conflict determination to an assertion of the execution environment's integrity at the time the determination was made.

Claim 13 — Three-Region Derivation (depends on Claim 1)

The method of Claim 1, wherein deriving the execution-authority state further comprises: when the maximum probability value is at or above the first threshold, issuing an attested execution-authority state authorizing action dispatch for the highest-ranked candidate; and when the maximum probability value is below the first threshold and the decision-quality metric is at or above the second threshold, issuing an attested execution-authority state authorizing action dispatch for the highest-ranked candidate with a mandatory reduced-confidence indicator; and wherein in all cases, the non-bypassable execution interface dispatches an action only upon receipt of an attested execution-authority state and never upon receipt of an inference engine output directly.

Claim 14 — Typed Authority-State Isolation (depends on Claim 1)

The method of Claim 1, wherein the non-bypassable execution interface accepts only authority-state tokens of a predefined, non-interpretable type, and wherein said authority-state tokens are cryptographically signed, contain no probability distribution, inference output, or derivative thereof, and are non-decodable into executable actions except through a fixed mapping table external to the inference engine, whereby the execution interface is type-restricted from processing any data originating from the inference engine.

Claim 15 — Non-Addressable Inference Memory Partition (depends on Claim 2)

The system of Claim 2, wherein the probability distribution and all inference outputs reside in a memory region that is not addressable by the non-bypassable execution interface, the memory region being isolated by a memory management unit such that page table entries for said region are absent from the address space of the execution interface process and any attempted access triggers a hardware-level fault, and wherein no process, thread, or module with

execution privileges has read access to said memory region, nor can request, reconstruct, or derive data from said region through inter-process communication, shared memory, direct memory access, or system calls, whereby no executable path exists from inference-state memory to execution-state memory.

Claim 16 — Structural Computational Isolation of Execution Interface (depends on Claim 2)

The system of Claim 2, wherein the non-bypassable execution interface is structurally absent the computational primitives necessary to perform arithmetic, statistical, or transformation operations on inference-domain data, the structural absence being enforced by at least one of the following mechanisms:

(a) ISA-level incapability: the execution interface comprises a fixed-function logicblock, field-programmable gate array, or application-specific integrated circuit that contains no general-purpose arithmetic logic unit, no floating-point processing unit, and no general-purpose instruction decoder, the logic block being physically limited to mapping input token types to output action commands through a read-only correspondence table, such that no instruction sequence executable by the logic block can perform floating-point arithmetic, integer multiplication, statistical aggregation, sorting, or any operation capable of computing a summary statistic from a probability distribution; or

(b) address-space-level structural exclusion: the execution interface executes within aprocess address space from which all inference-domain data — including the probability distribution, all inference engine outputs, and all data derived therefrom — is structurally excluded by a hardware-enforced memory isolation mechanism, such that no instruction executable by the execution interface can load, read, or access any inference-domain data into any register, cache line, or memory location within the execution interface's address space, and such that any attempted access to inference-domain memory triggers a non-maskable, non-

recoverable hardware fault; whereby the execution interface is structurally — not merely by policy, configuration, or software restriction — unable to derive or approximate actions from inference outputs, because the computational pathway from inference data to action emission does not exist within the execution interface's operational domain.

Claim 17 — Cryptographically One-Way Authority-State Transformation (depends on Claim 1)

The method of Claim 1, wherein the authority-state derivation layer produces authority-state tokens via a cryptographically one-way, irreversible transformation of system state, and wherein said transformation is non-invertible both structurally and mathematically, does not preserve sufficient information to reconstruct the probability distribution or any candidate action, and renders recovery of inference data from the authority-state token computationally infeasible, whereby action emission is decoupled from inference data at an information-theoretic level.

Claim 18 — Exclusive Execution Channel Enforcement (depends on Claim 2)

The system of Claim 2, wherein the non-bypassable execution interface is the sole hardware- or kernel-enforced pathway capable of issuing action commands, and wherein any alternative pathway is blocked at the operating system, firmware, or hardware interrupt level and cannot be instantiated without violating system integrity constraints, whereby side-channel or parallel execution paths are physically and logically precluded.

Claim 19 — Negative Attestation Dominance and State Destruction (depends on Claim 1)

The method of Claim 1, wherein upon entry into the conflict condition all transient computation states associated with inference — including all copies of the probability

distribution, intermediate activations, logits, pre-softmax values, and all buffers or caches used during inference and authority-state derivation — are irreversibly discarded through secure zeroization, and the negative attestation artifact becomes the only persistent system state accessible to all execution-capable components, whereby no residual or latent state exists that could be used to derive or approximate an action, and wherein said irreversible discard includes prevention of persistence or recoverability from transient computational artifacts, including processor registers, caches, buffers, speculative execution traces, and any intermediate representations, such that no residual or reconstructable decision representation remains accessible to any execution-capable component.

Claim 20 — Deterministic Replay Sufficiency Constraint (depends on Claim 1)

The method of Claim 1, wherein the negative attestation artifact contains the full probability distribution, threshold parameters, and evaluation metrics sufficient to reconstruct the conflict condition and to confirm that no executable action could have been produced, without access to the inference engine, the authority-state derivation layer, the model weights, the raw input data, or any internal system state.

SECTION IX — DISTINCTION OVER PRIOR ART

9.1 Distinction from Threshold-Based Classification Rejection (Chow 1970 and Progeny)

Chow (1970) established that a classifier may optimally reject inputs when the maximum posterior probability falls below a cost-derived threshold. Subsequent work extended this to margin-based, entropy-based, and learned rejection mechanisms, including SelectiveNet (Geifman and El-Yaniv, 2019) and the broader selective classification literature.

These methods share a common structural property: the rejection is internal to the classifier. The classifier outputs "reject" as one possible outcome. No structured attestation record is generated. No cryptographic signing occurs. No hash-chained audit log is maintained. No downstream action dispatch interface is gated on the rejection. No non-override invariant is enforced at the protocol level. The rejection is a classifier behavior, not a system-level control state.

ACHP-ADI differs structurally. The Authority Layer is a separate component from the inference engine. The Conflict State is structurally external to the inference engine's output space — the inference engine does not produce a "conflict" or "reject" label; rather, the Authority Layer independently determines that the inference engine's output distribution lacks sufficient quality. The Conflict Receipt is a negative attestation: it documents the absence of a decision, not the presence of one. No prior art rejection mechanism generates a structured, signed record documenting that a decision could not be made, containing sufficient data for deterministic replay of the decision boundary.

9.2 Distinction from Certified Control Systems (Jackson et al., MIT CSAIL, 2019–2020)

The "Certified Control" architecture for autonomous vehicles (Jackson, DeCastro, Kong, et al., CSAIL/Toyota Research Institute) employs an independent certifier that checks a

controller-generated certificate of safety before permitting action. When the certificate check fails, a safe fallback controller takes over. The certifier has prioritized access to actuators, achieving structural non-bypassability.

ACHP-ADI shares the structural pattern of an independent authority that gates action execution and activates fallback on rejection. However, the certified control architecture differs from ACHP-ADI in the following critical respects:

First, the certified control certifier evaluates physical safety conditions (obstacle geometry, lane compliance) based on sensor evidence. It does not evaluate the internal structure of a probability distribution for decision-margin sufficiency. The basis for the gating decision is qualitatively different.

Second, when the certified control certifier rejects a certificate, no structured attestation record is produced. The rejection is an event that triggers fallback, but no persistent, signed record documents what conditions led to the rejection or what evidence was insufficient. There is no Conflict Receipt and no negative attestation.

Third, the certified control architecture maintains no hash-chained, tamper-evident log of rejection events. There is no append-only chain of certifier rejections enabling post-hoc traversal.

Fourth, the certified control architecture provides no mechanism for deterministic replay of the decision boundary. An auditor cannot, after the fact, independently recompute the conditions under which the certifier rejected a certificate and verify that the rejection was correct. The certificates are checked at runtime and the check results are not preserved in a form enabling mathematical replay.

Fifth, the certified control architecture attests a positive condition: the certificate, when approved, constitutes evidence that the proposed action is safe. ACHP-ADI generates a negative

attestation: the Conflict Receipt documents that a decision could not be made. This is a structural inversion of the attestation direction.

9.3 Distinction from Cryptographic Audit Trails and Blockchain Logging

Recent work proposes blockchain-based and hash-chained logging of AI inference decisions on edge devices and in distributed systems. These systems log decisions that were made. They record the model's output and the action that was dispatched.

ACHP-ADI does not log decisions that were made. It generates a negative attestation of decisions that were refused. The Conflict Receipt documents the absence of a decision and provides the mathematical basis — through deterministic replay of the decision boundary — for an auditor to verify that the refusal was correct. Furthermore, the attestation is not post-hoc — it is a precondition for the system's control flow. The ADI does not log the conflict and then dispatch an action; it blocks the action because of the conflict, and the blocking occurs prior to and instead of action dispatch. The audit trail is a structural byproduct of the control mechanism, not an observational overlay on an existing decision pipeline.

9.4 Distinction from Attested Execution and TEE-Based Validation Systems

TEE-based remote attestation systems (Intel SGX, AMD SEV-SNP, Azure Attestation, IETF RATS Entity Attestation Token) require cryptographic attestation as a precondition for code execution or secret release. The attestation token is a signed, structured record. These systems share with ACHP-ADI the structural pattern of attestation-as-precondition.

However, TEE attestation systems attest a positive condition: that the execution environment is valid, that code measurements match expected values, that the platform is trustworthy. ACHP-ADI generates a negative attestation: that a decision could not be made due to insufficient margin in a probability distribution. The attestation domain is fundamentally

different: environment integrity versus probabilistic decision quality. TEE systems do not analyze probability distributions, compute decision margins, or produce attestation records specific to decision ambiguity. TEE systems gate on who or what is executing; ACHP-ADI gates on the quality of what was decided.

9.5 Distinction from Safety Interlocks and Industrial Control Systems

Safety interlocks in industrial and robotic systems prevent action execution when physical preconditions are not met (e.g., guard door open, pressure out of range). These interlocks operate on binary sensor states or threshold comparisons of physical quantities.

ACHP-ADI applies the interlock concept to probabilistic inference outputs. The "precondition" for action execution is not a physical sensor state but the internal structure of a probability distribution: sufficient confidence and sufficient margin between the top candidates. This is a qualitatively different input domain requiring analysis of a multi-valued distribution, not a single sensor reading. Additionally, safety interlocks do not produce structured negative attestation records or maintain hash-chained audit trails of interlock activations.

9.6 Distinction from Zero-Trust and Policy Enforcement Architectures

Zero-trust architectures enforce authorization requirements based on identity, credentials, context, and policy rules. The AI Governance and Accountability Protocol (AIGA) extends this to AI agent governance with kernel-level interception, tiered risk classification, and Merkle-root audit logs.

ACHP-ADI differs in the basis for its gating decision. Zero-trust systems gate on who is requesting and what they are requesting. ACHP-ADI gates on the quality of the probabilistic decision underlying the request. Zero-trust systems do not analyze probability distributions, compute decision margins, or produce structured negative attestation records of insufficient

decision quality. AIGA governs agent actions broadly; ACHP-ADI governs the specific transition from probabilistic inference to action dispatch based on decision-margin analysis.

9.7 Novelty as Conjunction

The novelty of ACHP-ADI does not reside in any single element. Threshold-based rejection is known. Cryptographic audit chains are known. Safety interlocks are known. Execution gating is known. Attestation-as-precondition is known. Fallback on failure is known.

The novelty resides in the specific integration of: (a) deterministic threshold-based partitioning of a probability distribution using a decision-quality metric into action-authorizing and non-action-authorizing regions; (b) explicit, structural refusal to emit any action-triggering output in the conflict region, where the conflict state is structurally external to the inference engine's output space; (c) generation of a negative attestation record — documenting that a decision could not be made — with sufficient content for deterministic replay of the decision boundary; (d) cryptographic signing and tamper-evident chaining of that record; (e) structural gating of action dispatch on the authority layer's prior determination, enforced by a non-bypassable action dispatch interface as the sole execution path; and (f) protocol-level non-override invariant preventing any downstream component from forcing an action for an input in the conflict state.

Each element is necessary. Removing any one element produces a materially different system that does not satisfy ACHP-ADI invariants. No identified prior art reference or combination discloses this specific conjunction operating as an integrated control architecture.

SECTION X — NON-GOALS

ACHP-ADI is not a confidence calibration method. It does not improve the accuracy of the inference engine's probability estimates.

ACHP-ADI is not a classification method. It does not produce classifications. It evaluates the quality of classifications produced by others and gates action execution accordingly.

ACHP-ADI does not replace human adjudication. It flags when adjudication is required and provides the cryptographic audit trail to support that adjudication.

ACHP-ADI does not resolve ambiguity. It generates a negative attestation of ambiguity and prevents ambiguity from causing uncontrolled action execution.

ACHP-ADI does not claim all forms of uncertainty quantification, classification rejection, or abstention. It claims the specific control architecture described herein.

SECTION XI — OPTIONAL EXTENSIONS

The following extensions do not alter the core ACHP-ADI architecture but expand its applicability:

Entropy Trigger. When enabled for a lane, a distributional ambiguity metric (such as Shannon entropy) of the probability distribution is computed. If the metric exceeds a lane-configurable threshold, the conflict state is entered regardless of the decision margin. This addresses distributions where the top two candidates have adequate margin but probability mass is spread thinly across many candidates. When this trigger activates the conflict state, the negative attestation record must indicate the triggering mechanism.

Human Attestation. After a Conflict Receipt is emitted, a human operator may provide an annotation (e.g., an inspection finding, a manual classification, a disposition decision). The annotation is cryptographically signed by the human operator and appended as a follow-up record linked to the original receipt's hash. The human attestation does not retroactively resolve the conflict state. The original receipt remains in the chain, and the non-override invariant is not affected. The human attestation is an addendum, not a correction.

Ensemble Aggregation. When the inference engine comprises multiple models, the probability distribution P may be computed as the arithmetic mean (or other specified aggregation function) of the individual model distributions before the Authority Layer evaluates P. The aggregation method is a lane configuration parameter.

Signing Failure Recovery. When the cryptographic signing module is unavailable (e.g., secure element powered down, TPM in error state), the receipt is stored with a pending-signature flag and signing is retried upon module availability. Unsigned receipts are flagged as unverified in any audit traversal. The ADI still blocks action dispatch and invokes fallback; the absence of a signature does not override the conflict determination.

SECTION XII — ABSTRACT

A control architecture (ACHP-ADI) for computing systems that process probabilistic inference outputs. The architecture comprises an independent Authority Layer that evaluates a probability distribution from an inference engine against fixed thresholds to determine whether the decision margin between top-ranked candidates is sufficient for action authorization. When margin is insufficient, the system enters a defined conflict state that is structurally external to the inference engine's output space, refuses to emit any action-triggering output, and generates a negative attestation — a cryptographically signed record documenting that a decision could not be made, containing the full distribution and decision metadata sufficient for deterministic replay of the decision boundary. The record is appended to a tamper-evident hash-chained log. All downstream action dispatch passes through a non-bypassable Action Dispatch Interface that permits action only upon prior authorization from the Authority Layer, blocks primary action paths on conflict, and invokes fallback behavior. The architecture converts probabilistic uncertainty into a verifiable control state through negative attestation, enabling independent verification that ambiguous decisions never caused uncontrolled action execution.

SECTION XIII — ENFORCEMENT ARCHITECTURE AND HARDWARE-BACKED

ISOLATION

This section describes implementation-level mechanisms that support the enforcement properties required by the claims. These mechanisms are not limiting; the claims are defined by their structural requirements, not by any particular implementation. The descriptions below illustrate how a conforming implementation may satisfy the enforcement requirements using available hardware and software primitives.

13.1 Memory Isolation Enforcement (Supporting Claim 15)

In one embodiment, the system partitions physical memory into at least two non-overlapping regions enforced by the processor's memory management unit (MMU).

A first region, the inference memory region, stores the probability distribution, all intermediate inference computations, model weights, activations, and any data produced by or derived from the inference engine. The inference memory region is mapped into the virtual address space of the inference engine process only. The page table entries for this region do not appear in and are not accessible from the page tables of the execution interface process, the authority-state derivation process (after token emission), or any action executor process. The MMU enforces these partitions at the hardware level: page table entries for the inference memory region carry permission bits that exclude read, write, and execute access from all processes other than the inference engine. Attempted access from the execution interface process triggers a hardware-level page fault that is non-maskable and non-recoverable by the faulting process.

A second region, the execution memory region, stores attested execution-authority states, the authority-state token buffer, and action command structures. The execution memory region is mapped into the virtual address space of the execution interface process only.

In some implementations, the authority-state derivation layer operates with time-limited read access to the inference memory region (to compute decision metrics) and write access to the execution memory region (to emit authority-state tokens). Upon completion of authority-state derivation for a given input, the kernel revokes the authority-state derivation layer's read access to the inference memory region for that input's data by modifying page table entries and flushing the translation lookaside buffer (TLB), preventing subsequent access.

In some implementations, no system call, inter-process communication channel, shared memory mapping, or direct memory access (DMA) transfer permits the execution interface to access the inference memory region. DMA controllers are configured with I/O MMU (IOMMU) restrictions that prevent DMA-capable peripherals associated with the execution interface from accessing inference memory address ranges.

In some implementations, inference memory resides within a trusted execution environment (TEE) or secure enclave such that even privileged kernel processes cannot access it. On platforms supporting Intel SGX, the inference memory region is enclosed within an enclave. Data within the enclave is encrypted in main memory by the processor's memory encryption engine and is accessible only to code executing within the enclave. The execution interface executes outside the enclave and cannot access enclave memory; attempted access returns encrypted ciphertext that is computationally infeasible to decrypt without the enclave's internal keys. On platforms supporting ARM TrustZone, the inference memory resides in the secure world partition, which is hardware-isolated from the normal world in which the execution interface operates. The TrustZone address space controller (TZASC) blocks all normal-world access to secure-world memory at the bus level.

13.2 Structural Computational Isolation of Execution Interface (Supporting Claim 16)

The claimed system requires that the execution interface be structurally absent the computational primitives necessary to operate on inference-domain data. This section

describes two categories of structural enforcement, each independently sufficient to satisfy the claim. In both categories, the limitation is structural — it arises from the physical or architectural composition of the execution interface — and is not a software policy that could be overridden, reconfigured, or bypassed.

Category A — ISA-Level Incapability (Hardware Embodiments)

In one embodiment, the execution interface is implemented as a fixed-function logic block (e.g., an FPGA module or ASIC) that accepts typed authority-state tokens on a hardware input bus and emits action commands on a separate hardware output bus. The logic block contains: a token type register that reads the type identifier from incoming tokens; a signature-verification module (which may be a dedicated cryptographic coprocessor or a hardwired comparison circuit); a read-only memory (ROM) correspondence table that maps token type identifiers to action command codes; and an output register that transmits the mapped action command.

The logic block does not contain: a general-purpose arithmetic logic unit (ALU); a floating-point processing unit (FPU); a general-purpose instruction decoder; a program counter or instruction fetch unit; integer multiplication or division hardware; comparison circuitry operating on variable-length arrays; or any hardware capable of computing argmax, mean, variance, entropy, sum, sort, or any statistical aggregate.

The logic block is physically incapable of performing these operations. The incapability does not depend on software configuration, firmware state, or operating system policy. No firmware update, configuration change, or privilege escalation can introduce arithmetic or statistical capability into the logic block, because the necessary transistor-level circuitry does not exist.

In some implementations, the logic block is fabricated as a mask-programmed ASIC, and the ROM correspondence table is immutable after fabrication. In some implementations,

the logic block is implemented in an FPGA with a write-locked configuration bitstream, and the bitstream is cryptographically signed by the manufacturer and cannot be reprogrammed in the field without hardware key material.

Category B — Address-Space-Level Structural Exclusion (Software-on-Hardware Embodiments)

In one embodiment, the execution interface executes as a software process on a general-purpose processor, but operates within an address space from which all inference-domain data is structurally excluded by hardware-enforced memory isolation.

The structural exclusion operates as follows. The processor's memory management unit (MMU) maintains separate page tables for the execution interface process and the inference engine process. The page table entries for the inference memory region — containing the probability distribution, all intermediate inference computations, model weights, activations, and all data produced by or derived from the inference engine — are absent from the execution interface's page tables. The inference memory region does not appear in the execution interface's virtual address space. It is not mapped as readable, writable, or executable. It is not mapped at all.

Any attempt by the execution interface process to access a virtual address corresponding to the inference memory region will fail, because that virtual address does not resolve to any physical page in the execution interface's page table. The MMU generates a hardware page fault. This fault is non-maskable (it cannot be suppressed by the execution interface process) and non-recoverable (the faulting instruction cannot be retried with different permissions). No system call, inter-process communication channel, shared memory mapping, or DMA transfer can introduce inference-domain data into the execution interface's address space, because the kernel enforces the page table separation and the IOMMU restricts DMA-capable peripherals.

The structural nature of this exclusion is as follows: even though the underlying processor possesses a general-purpose ALU and FPU, these units cannot operate on inference-domain data because inference-domain data cannot be loaded into any register accessible to the execution interface. An arithmetic instruction requires operands in registers. Registers are loaded from memory. The memory containing inference data is not addressable by the execution interface. Therefore, no sequence of instructions executable by the execution interface — regardless of the instruction set available — can load, access, or operate upon inference-domain data. The incapability is not that the processor lacks arithmetic instructions; the incapability is that there exists no instruction sequence that can bring inference-domain data into a state where arithmetic instructions can reach it.

This is a structural limitation, not a policy limitation. A policy limitation (e.g., "the process is forbidden from accessing memory region X") could be overridden by a privileged process modifying the policy. In this implementation, the page table entries for the inference memory region do not exist in the execution interface's address space, the kernel enforces this separation through mandatory access controls that are not overridable by the execution interface process, and the MMU enforces it at the hardware level by generating faults on unmapped addresses. The inference data is not "forbidden" to the execution interface — it is structurally absent from the execution interface's operational domain.

In some implementations, the inference memory resides within a TEE or secure enclave such that even privileged kernel processes cannot access it. On platforms supporting Intel SGX, the inference memory is encrypted in main memory and decryptable only by code executing within the enclave. On platforms supporting ARM TrustZone, the TrustZone address space controller (TZASC) blocks all normal-world bus transactions targeting secure-world memory at the hardware bus level.

Convergence of Both Categories

Both Category A and Category B achieve the same structural result: no computational pathway from inference-domain data to action emission exists within the execution interface's operational domain. In Category A, the pathway does not exist because the execution interface lacks the computational machinery to process the data. In Category B, the pathway does not exist because the data cannot enter any location where the computational machinery could reach it. In both cases, the absence is structural and hardware-enforced, not behavioral or policy-based.

The authority-state tokens accepted by the execution interface in both categories are strictly typed. Each token conforms to a predefined schema containing: a token type identifier (from a fixed enumeration), a cryptographic signature, a timestamp, and a lane identifier. The token does not contain the probability distribution, any probability value, any inference engine output, or any data derived from the probability distribution. The token type identifier maps to a specific action command through the fixed correspondence table. The execution interface reads only the type identifier. The absence of inference data in the token ensures that even the token-processing path within the execution interface never encounters inference-domain data.

In the present system, non-instantiation is enforced not only at the level of stored or transmitted data, but across the entire computational lifecycle. This includes prevention of decision representation formation within transient or intermediate machine states such as CPU registers, cache hierarchies, pipeline stages, speculative execution paths, or temporary buffers. The system is architected such that no executable, classifiable, or derivable decision object is ever instantiated, even momentarily, prior to, during, or after authority-state derivation. This distinguishes the system from architectures in which a decision is formed and subsequently filtered, rejected, or withheld. In those architectures, a decision object exists — transiently or persistently — and a downstream component acts upon it. In the present system, no decision object comes into existence at any stage. The authority-state derivation layer consumes the

probability distribution and produces either an attested execution-authority state or a negative attestation artifact. At no point in this transformation is a candidate action, class label, or executable decision instantiated as a data object, register value, or intermediate computation product. The probability distribution is evaluated against thresholds; the evaluation yields a region determination; the region determination is encoded directly into the authority-state token or negative attestation artifact. No intermediate step produces a "proposed action" that is subsequently approved or blocked.

This architecture is distinct from systems in which a decision is formed and subsequently filtered, rejected, or withheld. In such systems, a decision object necessarily exists, even if only transiently, within registers, buffers, or intermediate computational states. In contrast, the present system is configured such that no executable, classifiable, or derivable decision representation is instantiated at any stage of computation, including transient and speculative states. This distinction reflects a topological constraint on machine operation rather than a post hoc control over data that has already been created.

13.3 Kernel and Execution Pathway Control (Supporting Claim 18)

In one embodiment, the operating system kernel or firmware enforces that the execution interface is the sole pathway capable of issuing action commands to action executors.

Action executors (e.g., motor controllers, API response handlers, actuator drivers) are registered with the kernel as protected resources. The kernel maintains a mandatory access control policy that permits only the execution interface process to send commands to these resources. All other processes, including the inference engine process and the authority-state derivation process, are denied write access to action executor resources. This enforcement operates at the kernel level and cannot be overridden by user-space processes regardless of privilege level.

In some implementations, system call filtering (e.g., seccomp-bpf on Linux, pledge on OpenBSD) is applied to all processes other than the execution interface, prohibiting system calls that could directly or indirectly transmit commands to action executors. The filter blocks: direct I/O writes to hardware device registers (ioperm, iopl, /dev/mem); ioctl calls to actuator device drivers; network socket operations targeting action executor endpoints; creation of new processes or threads with access to action executor resources; and memory mapping of device-mapped I/O regions associated with action executors.

In some implementations, the hardware interrupt controller is configured so that interrupt lines connected to action executor hardware are serviceable only by the execution interface's interrupt handler. Interrupt requests originating from any other process context are masked at the interrupt controller level and discarded.

In some implementations on embedded or real-time platforms, the execution interface is implemented in a separate hardware partition (e.g., a dedicated processor core, a separate microcontroller, or a hardware security module) with a dedicated bus connection to action executors. No bus arbitration path exists between the inference engine's processor and the action executor bus. Physical isolation at the bus level ensures that no software-level bypass is possible. On multi-core systems, core affinity and hardware partitioning (e.g., ARM TrustZone-based partitioning of bus masters) ensure that the inference engine core cannot issue transactions on the action executor bus.

In some implementations, the firmware boot sequence configures the execution pathway enforcement before the operating system loads, and the configuration is locked against runtime modification by setting hardware configuration lock bits. Subsequent software, including the operating system kernel, cannot alter the bus routing, interrupt assignments, or memory protection settings that enforce the exclusive execution channel.

13.4 One-Way Authority-State Transformation (Supporting Claim 17)

In one embodiment, the authority-state derivation layer produces authority-state tokens via a cryptographically one-way, irreversible transformation that is non-invertible both structurally and mathematically and does not preserve sufficient information to reconstruct the probability distribution or any candidate action. Reconstruction of the probability distribution or any candidate action from the authority-state token is computationally infeasible.

The transformation proceeds as follows. The authority-state derivation layer evaluates the probability distribution against the configured thresholds and determines the decision region (Strong, Weak, or Conflict). For the Strong and Weak regions, the layer emits a token containing: the decision region identifier; the index of the top-ranked candidate (but not the probability values); a confidence indicator (full or reduced, but not the numerical confidence value); a lane identifier; a timestamp; and a cryptographic signature over the token contents. The token does not contain the probability distribution, the individual probability values, the decision-quality metric value, or any data from which these could be recovered.

The non-invertibility of the transformation is enforced through two independent mechanisms. First, the token schema is structurally non-invertible: the schema contains no fields for probability data, so the transformation discards the probability values entirely. There is no encoding of the probability distribution in any token field, and no combination of token fields can be used to reconstruct the distribution. Second, where audit linkage is required, the authority-state derivation layer applies a cryptographic hash function (e.g., SHA-256, SHA-3) to the probability distribution and includes only the resulting hash digest in the token. Cryptographic hash functions are computationally one-way: given a hash digest, computing a pre-image (the probability distribution that produced it) requires effort that is exponential in the hash output length and is considered computationally infeasible under standard cryptographic assumptions.

In some implementations, the authority-state derivation layer employs an irreversible transformation function that maps the probability distribution to a fixed-size output via a keyed hash-based message authentication code (HMAC) with a device-resident key. The HMAC output serves as the audit linkage identifier. The key is stored in a hardware secure element and is not accessible to the execution interface or any action executor. Even with access to the HMAC output, reconstruction of the input distribution is computationally infeasible without the key, and infeasible with the key due to the irreversible many-to-one nature of the hash function.

For the Conflict region, no authority-state token authorizing action is emitted. The negative attestation artifact is produced, which does contain the full probability distribution for replay purposes. The execution interface does not extract or use the probability data within the negative attestation artifact; it responds only to the artifact's type (negative attestation) and status indicator to trigger blocking and fallback. The probability data within the artifact persists solely in the tamper-evident log for post-hoc replay.

13.5 Irreversible State Discard (Supporting Claim 19)

In one embodiment, upon entry into the conflict condition, all transient computation states associated with the inference evaluation for the corresponding input are irreversibly discarded through secure zeroization.

The discard procedure operates as follows. After the authority-state derivation layer has computed the decision metrics and determined that the conflict region has been entered, and after the negative attestation artifact has been constructed and signed, the system executes a secure zeroization of all memory locations containing: the probability distribution for the corresponding input; all intermediate values computed during inference (activations, logits, pre-softmax values, attention weights, embedding vectors); the computed decision-quality metric, maximum probability value, and second-largest probability value (these values persist only

within the signed negative attestation artifact in the tamper-evident log); and any buffers, caches, registers, or temporary storage used during the inference and authority-state derivation computation for the corresponding input.

In some implementations, the zeroization is performed by the kernel or a trusted zeroization module that overwrites all affected memory locations with zero values, then overwrites with cryptographically random values, then overwrites with zero values again (three-pass overwrite). The zeroization is performed synchronously before the system enters fallback mode, ensuring that no residual state survives the transition to fallback. The zeroization completion is confirmed by a read-back pass that compares the overwritten region against the expected zero pattern.

In some implementations on hardware supporting Intel SGX, the enclave containing the inference data is destroyed via EREMOVE instructions for all enclave pages, followed by an EBLOCK and EWB sequence that evicts and destroys the enclave's memory contents. The hardware ensures that the encryption keys used for enclave memory are discarded upon teardown, rendering the memory contents unrecoverable even through physical memory inspection.

In some implementations on hardware supporting ARM TrustZone, the secure world context is terminated and the secure world memory is scrubbed by the trusted firmware before control returns to the normal world. The TrustZone address space controller is reconfigured to release the secure memory region, but only after the scrub has completed and the memory contents are confirmed to be zero.

In some implementations, the inference engine operates on a dedicated volatile memory region (e.g., battery-backed SRAM) with a hardware-backed erase function. The erase function is triggered by a hardware signal from the authority-state derivation layer upon conflict

determination. The erase completes in bounded time (specified per the hardware datasheet) before fallback invocation.

The negative attestation artifact, which contains the probability distribution and metrics, persists in the tamper-evident log in non-volatile memory. This is by design: the artifact must persist for deterministic replay. The distinction is that the artifact is the exclusive persistent representation of the system state for the corresponding input, and all other copies of the inference data are destroyed. No component other than an auditor performing replay accesses the probability data within the artifact.

Zeroization includes active invalidation and overwrite of transient computational artifacts and ensures that no inference-derived values capable of representing or reconstructing a decision persist in any physical or virtual memory layer, including registers, caches, or speculative execution structures. In some implementations, the zeroization procedure issues explicit register-clearing instructions (e.g., XOR of general-purpose and floating-point registers to zero) following completion of authority-state derivation, ensuring that no inference-derived value remains in any architectural register. In some implementations on processors with speculative execution, the zeroization procedure includes a speculation barrier instruction (e.g., LFENCE on x86, CSDB on ARM) prior to the transition to fallback mode, ensuring that no speculatively loaded inference data remains in the pipeline or micro-architectural buffers. In some implementations, the cache lines associated with the inference memory region are explicitly invalidated (e.g., via CLFLUSH or DC CIVAC instructions) as part of the zeroization procedure, ensuring that no inference data persists in any level of the cache hierarchy after discard.

13.6 Deterministic Replay Sufficiency (Supporting Claim 20)

In one embodiment, the negative attestation artifact is structured to be self-contained for deterministic replay of the decision boundary.

The artifact contains, at minimum: the full probability distribution P as an ordered array of floating-point values; the applied confidence threshold (T_decide) and separation threshold (T_sep); the computed maximum probability value (p_max); the computed second-largest probability value (p_second); and the computed decision-quality metric (decision margin = p_max minus p_second).

An independent party performing replay requires no access to the inference engine, the authority-state derivation layer, the model weights, the raw input data, or any internal system state. The replay procedure is: (1) extract P, T_decide, and T_sep from the artifact; (2) compute max(P) and confirm it equals the recorded p_max; (3) compute the second-largest value in P and confirm it equals the recorded p_second; (4) compute the decision margin and confirm it equals the recorded metric; (5) confirm that p_max is less than T_decide and the decision margin is less than T_sep, thereby confirming that the conflict region was correctly entered.

In some implementations, the replay procedure is encoded as a standalone function that accepts a single artifact as input and returns a Boolean result. The function has no external dependencies, no state, and no side effects. It can be executed on any platform with floating-point arithmetic support.

In some implementations, the artifact also contains: the cryptographic hash of the raw input data (for cross-referencing with input logs, but not required for decision-boundary replay); the lane identifier and threshold configuration version (for audit trail linkage); and optional supplementary metrics (entropy, global spread) for forensic analysis. These additional fields support broader audit functions but are not required for the core decision-boundary replay.

13.7 Machine-Level Non-Instantiation and Hardware-Constrained Execution Topology

The system is directed to a specific machine configuration that enforces the non-existence of executable decision representations across all stages of computation. This includes

not only persistent memory structures, but also transient, intermediate, cached, register-level, speculative, and ephemeral machine states.

Unlike conventional computing systems in which data is generated and subsequently controlled, filtered, or protected, the present system is architected such that a class of data structures — namely executable, classifiable, or derivable decision representations — is never instantiated within the machine. This condition is enforced structurally through the absence of computational pathways capable of transforming inference-domain data into action-domain representations.

The execution interface is configured such that no data path exists from inference outputs, probability distributions, or intermediate computational artifacts to action emission. The execution interface receives only an execution-authority state that is cryptographically derived and non-invertible, and which contains no reconstructable information from the originating inference domain. As a result, no executable decision object can be formed, transmitted, or reconstructed within or through the execution interface.

This constraint is not a policy or rule applied to data that exists. It is a structural property of the machine in which the relevant class of data is rendered non-existent by design. The system therefore modifies the operation of the computing device itself by constraining the set of representable machine states.

At the micro-architectural level, this non-instantiation property is enforced through mechanisms including but not limited to register clearing, speculative execution barriers, cache invalidation, and memory isolation. These mechanisms ensure that no residual, transient, or speculative representation of a decision object can exist or be recovered from processor registers, execution pipelines, cache hierarchies, or memory subsystems.

SECTION XIV — PRACTICAL APPLICATION AND IMPROVEMENT TO COMPUTER FUNCTIONALITY

Conventional computing systems assume the existence of data objects and apply mechanisms such as encryption, access control, logging, or validation to manage those objects. In contrast, the present system introduces a different paradigm in which certain classes of data objects are structurally prevented from existing within the machine. This is accomplished through the elimination of computational pathways, enforcement of memory isolation boundaries, and control of micro-architectural state transitions.

This approach represents a specific and non-conventional improvement to computer functionality. It changes how the machine operates by constraining which data representations can be formed, rather than by regulating how existing data is used. The resulting system is therefore not an implementation of an abstract idea on a generic computer, but a reconfiguration of the computer's operational topology.

End of Provisional Patent Application Specification.